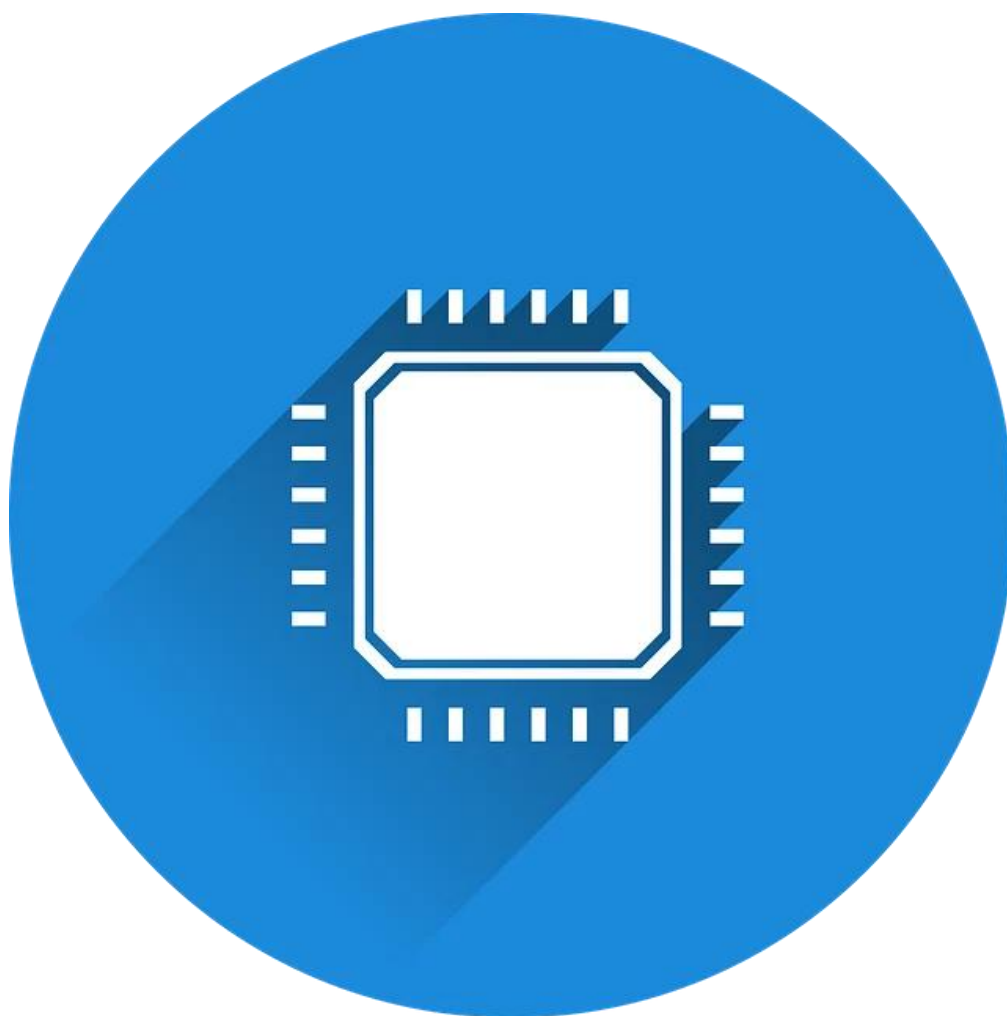




Device: ZIRCON-series CPU

Engineer: Zidane Hadeed

Contents

Abstract	ii
List of Figures	iii
List of Tables	iv
List of Abbreviations	v
1 Design Algorithm.....	6
2 System Requirements Specification	7
3 System Architecture Design	8
3.1 CODEC Architecture Design.....	11
3.2 GPR Architecture Design	13
3.3 ALU Architecture Design	15
3.3.1 Ripple Adder Architecture Design.....	17
3.3.2 Shifter Architecture Design	18
3.4 L1-D Cache Architecture Design.....	19
4 System Test Plan.....	21
4.1 SRS REQ3-1	22
4.2 SRS REQ3-2	24
4.3 SRS REQ3-3	26
4.4 SRS REQ3-4, REQ3-5, and REQ3-6.....	28
5 Results.....	31
5.1 Post System Test Plan.....	32
5.2 System Test Plan Summary	37
5.3 Requirements Traceability Matrix	38
6 Conclusion	40
References.....	41
Appendix A – Extracts of RISC-V Unprivileged ISA.....	42
Appendix B – Control Matrix	44
Appendix C – Zircon Assembler v1.0.0	45

Abstract

A CPU is responsible for the fetching, decoding, executing, and memory management of instructions in a computing device. It makes decisions by performing mathematical and logical operations between registers, memory, and literals. They run a set number of rudimentary instructions based on their supported ISA. And combinations of these rudimentary instructions form pseudo-instructions in more abstract high level programming languages. This project sought to design and implement a single core, dual cycle 32-bit CPU for educational use based on the unprivileged RISC-V RV32I ISA. This device called Zircon is functionally required to perform basic arithmetic and logic, conditionals, calls, returns, and NOPs. The RTL design was implemented with VHDL. It was then synthesised on AMD Vivado Design Suite 2022.2 and simulated on XSim.

List of Figures

Figure 1: Design Algorithm	6
Figure 2: Dual-Cyle Operation	8
Figure 3: CPU Architecture Datapath Design	10
Figure 4: CODEC Architecture Datapath Design.....	12
Figure 5: GPR and PC Structure (Source: RISC-V International 2026)	13
Figure 6: GPR File Architecture	14
Figure 7: ALU Architecture.....	16
Figure 8: Ripple Adder Architecture	17
Figure 9: Shifter Architecture	18
Figure 10: (a) Byte-Addressable Memory and (b) Big- and Little-Endian Addressing (Source: Harris and Harris 2012)	19
Figure 11: L1-D Cache Architecture	20
Figure 12: Hardware Utilisation	31
Figure 13: Power Utilisation.....	31
Figure 14: (TC3-1) Reads, Writes, Loads, and Stores.....	32
Figure 15: (TC3-2) Basic Logic.....	33
Figure 16: (TC3-3) Basic Arithmetic.....	34
Figure 17: (TC3-4/6) Branch Conditionals and NOPs	35
Figure 18: (TC3-5/6) Call, Return, and NOP	36
Figure C-1: ZASM v1.0.0 Algorithm	45

List of Tables

Table 1: Functional Requirements	7
Table 2: Decoder Control Logic	11
Table 3: ALU Control Logic.....	15
Table 4: STP Abbreviations	21
Table 5: STP Technical Terms.....	21
Table 6: SRS REQ3-1 Description	22
Table 7: Test Case (TC3-1) Inputs.....	22
Table 8: Test Case (TC3-2) Desired Outputs.....	23
Table 9: SRS REQ3-2 Description	24
Table 10: Test Case (TC3-2) Inputs.....	24
Table 11: Test Case (TC3-2) Desired Outputs.....	25
Table 12: SRS REQ3-3 Description	26
Table 13: Test Case (TC3-3) Inputs.....	26
Table 14: Test Case (TC3-3) Desired Outputs.....	27
Table 15: SRS REQ3-4, -5, -6 Description.....	28
Table 16: Test Case (TC3-4) Inputs.....	29
Table 17: Test Case (TC3-5) Inputs.....	29
Table 18: Test Case (TC3-6) Inputs.....	29
Table 19: Test Case (TC3-4) Desired Outputs.....	30
Table 20: Test Case (TC3-5) Desired Outputs.....	30
Table 21: Test Case (TC3-6) Desired Outputs.....	30
Table 22: System Test Plan Summary	37
Table 23: Requirements Traceability Matrix	38
Table 24: Requirements Status Monitoring	39
Table B-1: Control Matrix	44

List of Abbreviations

ALU	Arithmetic Logic Unit
CODEC	Controller-Decoder
CPU	Central Processing Unit
DEMUX	Demultiplex(er)
ISA	Instruction Set Architecture
GPR	General Purpose Register
LSB	Least Significant Bit
LUT	Look-up Table
MAC	Memory Access Controller
MSB	Most Significant Bit
MUX	Multiplex(er)
RAM	Random Access Memory
REG	Register
ROM	Read Only Memory

1 Design Algorithm

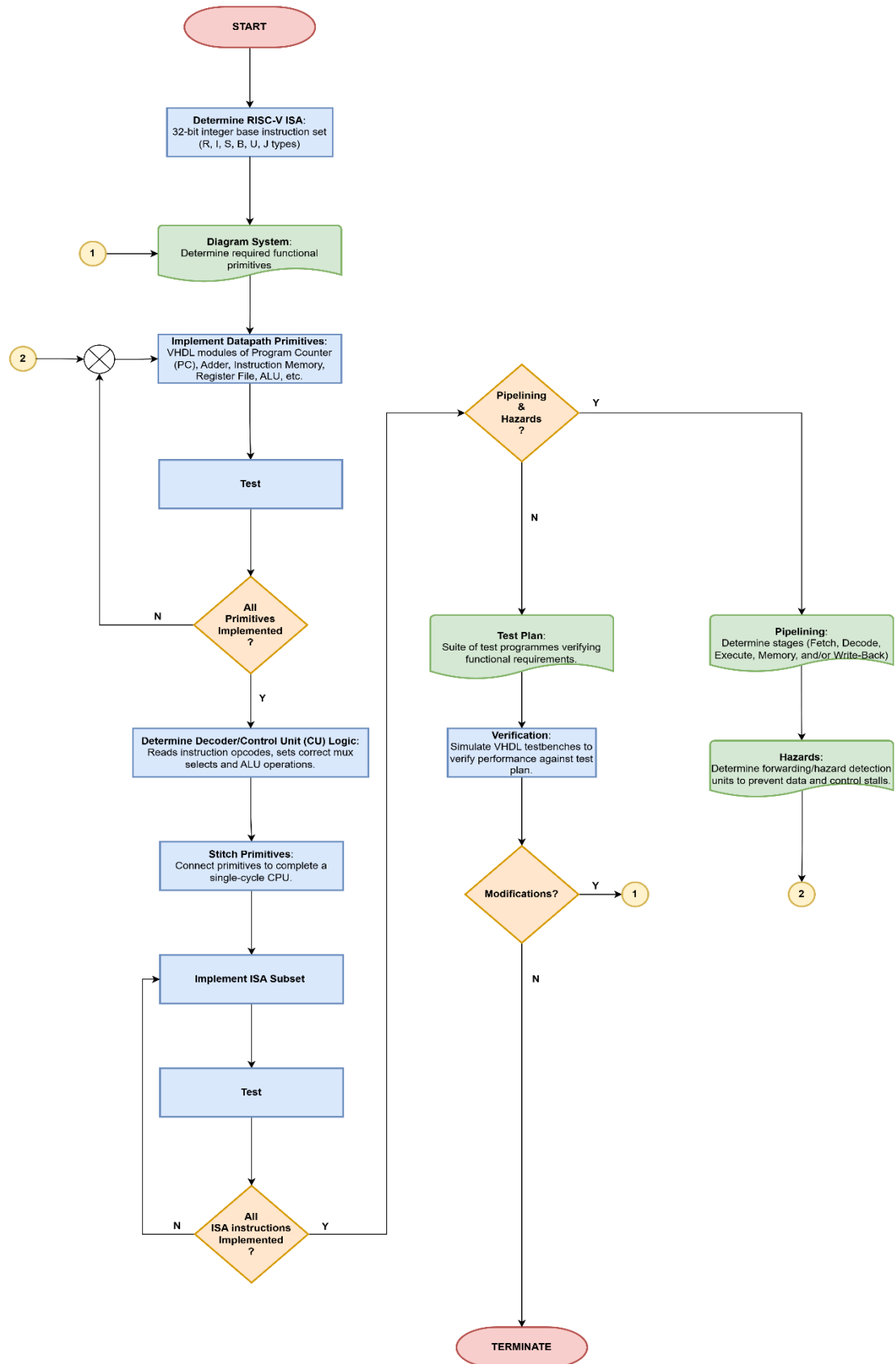


Figure 1: Design Algorithm

2 System Requirements Specification

Version: v1.0.0

Date: 18/05/26

Zircon is an educational 32-bit RISC-V CPU series that is capable of basic integer arithmetic, logical operations, shifts, comparisons, and no operation. Table 1 summarises its functional requirements.

Table 1: Functional Requirements

Code	Requirement
REQ1-1	Dual-issue, single-core
REQ2-1	2-stage in-order pipeline
REQ3-1	RV32I ISA
<i>REQ3-1.1</i>	<i>Read</i>
<i>REQ3-1.2</i>	<i>Write</i>
<i>REQ3-1.3</i>	<i>Load</i>
<i>REQ3-1.4</i>	<i>Store</i>
REQ3-2	Basic Logic
<i>REQ3-2.1</i>	<i>Bitwise OR</i>
<i>REQ3-2.2</i>	<i>Bitwise XOR</i>
<i>REQ3-2.3</i>	<i>Bitwise AND</i>
<i>REQ3-2.4</i>	<i>Bitwise NOT</i>
<i>REQ3-2.5</i>	<i>Left Shift</i>
<i>REQ3-2.6</i>	<i>Right Shift</i>
REQ3-3	Basic Arithmetic
<i>REQ3-3.1</i>	<i>Add</i>
<i>REQ3-3.2</i>	<i>Subtract</i>
<i>REQ3-3.3</i>	<i>Compare</i>
<i>REQ3-3.4</i>	<i>Right Shift</i>
REQ3-4	Branch Conditional
REQ3-5	Jump
REQ3-6	No Operation (NOP)

3 System Architecture Design

Version: v1.0.0

Date: 23/06/26

The datapath design of the overall system is illustrated in Figure 3. Its primary components are PC, Instruction ROM, CODEC, GPR, ALU, and Data RAM. PC is a register which stores the memory address of the currently executed instruction. Outputs of PC are passed with an offset through an Adder, which obtains the memory address of a sequential or branch instruction. Outputs of PC are also passed with an offset through the ALU, to obtain target addresses of both Jump and Return instructions. These addresses are MUXed, then asserted onto the input of PC, which will be read and stored at each clock cycle.

Instruction ROM is an asynchronous addressable memory unit. It retrieves internally stored 8-byte instructions which are flashed by an assembled binary file. Instructions are fetched from a memory address pointed by the PC once the unit enabled. The CODEC block asynchronously reads a single instruction word and determines its validity. An exception code and flag will be raised if an illegal instruction is determined. Instruction data are extracted and control signals asserted if otherwise.

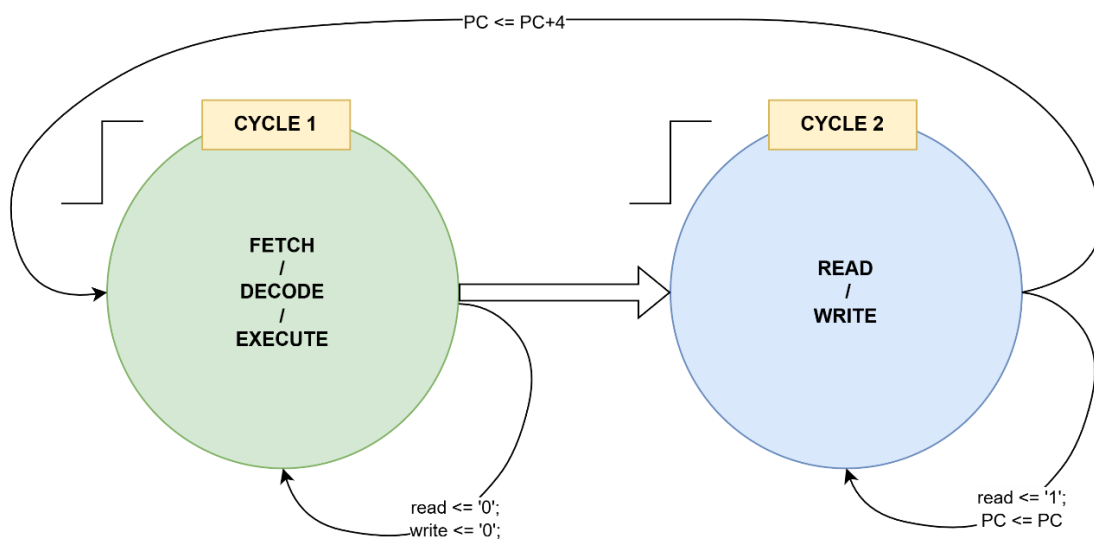


Figure 2: Dual-Cycle Operation

Instruction data consists of source, destination, and immediate values. Source and destination values index the location of registers within the GPR file. Destination indexed registers store the result of an operation at the Data port during write_enable (wen). The GPR also outputs source indexed registers as operands during read_enable (ren). Each operand is MUXed with the current instruction address and an immediate value. These are passed through the ALU which performs either an arithmetic or logical operation. The result is then written to the GPR file, or forms either a target or Data RAM address.

Data RAM is a byte-addressable memory unit that stores or loads data at a location indexed at its Address port. Store operations are executed within a single clock cycle from write assertion. Load operations require an extra clock cycle between read assertion and data output. This arises from the inherent latency of searching the RAM array, which requires dual cycle operation as in Figure 2. Zircon detects a load operation and then halts PC increments and ROM fetches. This works by injecting bubbles down their enable lines, which stalls both units by one clock cycle using a REG. This ceases after removal of the read signal and self-resets for proceeding loads.

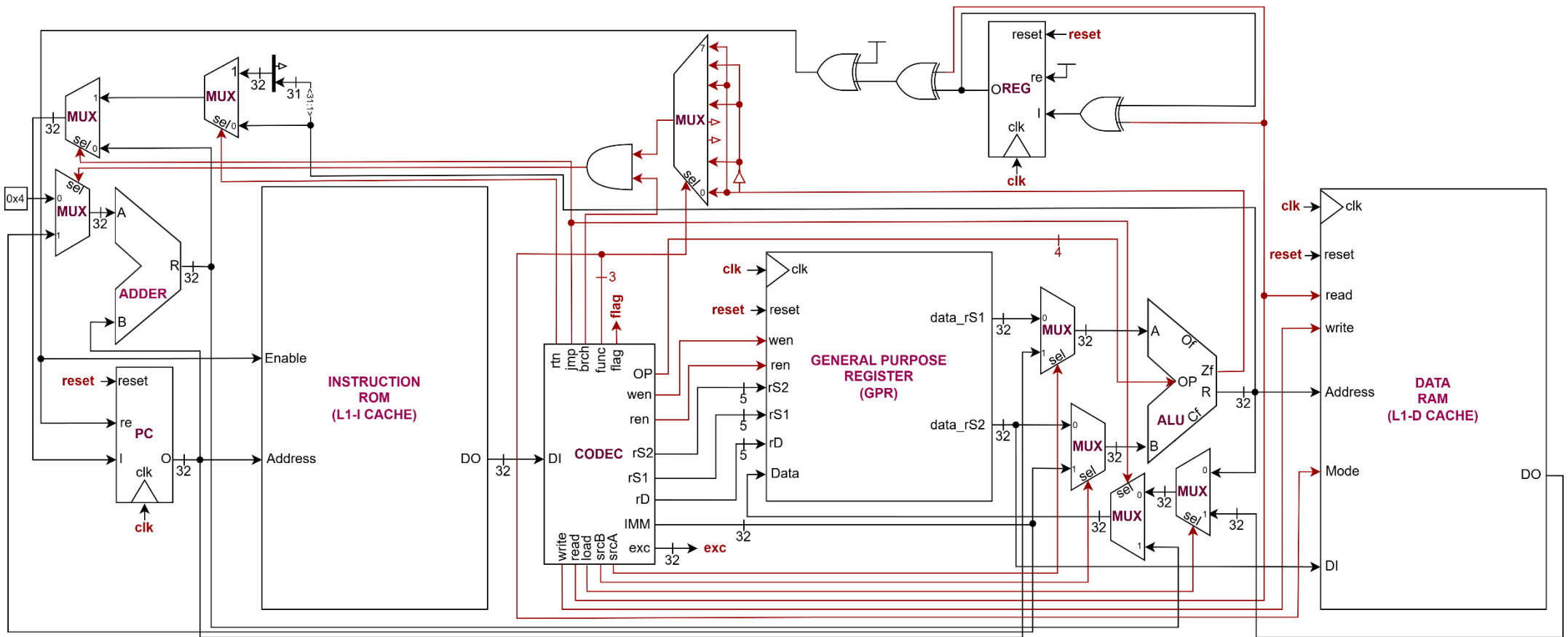


Figure 3: CPU Architecture Datapath Design

3.1 CODEC Architecture Design

RISC-V International (2026) categorises and formats instructions of the RV32I ISA by operation as per Appendix A (RISC-V International 2026), where –

- R-type: register-to-register integer computation;
- I-type: immediate and load integer computation;
- S-type: storage;
- B-type: conditional branch;
- U-type: special long immediate; and
- J-type: unconditional jumps.

Table 2: CODEC Logic

Instruction		Control Logic										
Type	Opcode(s)	sel	ren	wen	rtn	jmp	brch	write	read	load	srcA	srcB
R	0110011	000	1	1	0	0	0	0	0	0	0	0
I	0000011 1100111 0010011 0001111 1110011	010	1	1	X	X	0	0	X	X	0	1
S	0100011	011	1	0	0	0	0	1	0	0	0	1
B	1100011	001	1	0	0	0	1	0	0	0	0	0
U	0110111 0010111	100	0	1	0	0	0	0	0	0	X	1
J	1101111	101	0	1	0	1	0	0	0	0	1	1

Figure 4 illustrates the design of the CODEC block. It is responsible for translating machine-coded instructions based on the syntax of RV32I. A master decoder first determines the instruction type. It then selects an appropriate slave decoder by grounding its inhibit port. Data relevant to each instruction is then extracted based on the specific encoding format of the instruction. Instructions containing immediate values are further sign extended to 32-bits as required. These instruction data are each assigned an input channel to a relevant port MUX. Selection of input channels is controlled by the select bus from the master decoder. Table 2 gives the control logic for each instruction type. All ports are pulled LOW by default for no- and illegal- instructions. Illegal instructions raise a flag with exception code 0xFFFF. Opcodes b“0001111” and b“1110011” are unimplemented but raise exception code 0xAAAA. Table B-1 summarises the control matrix of all supported instructions.

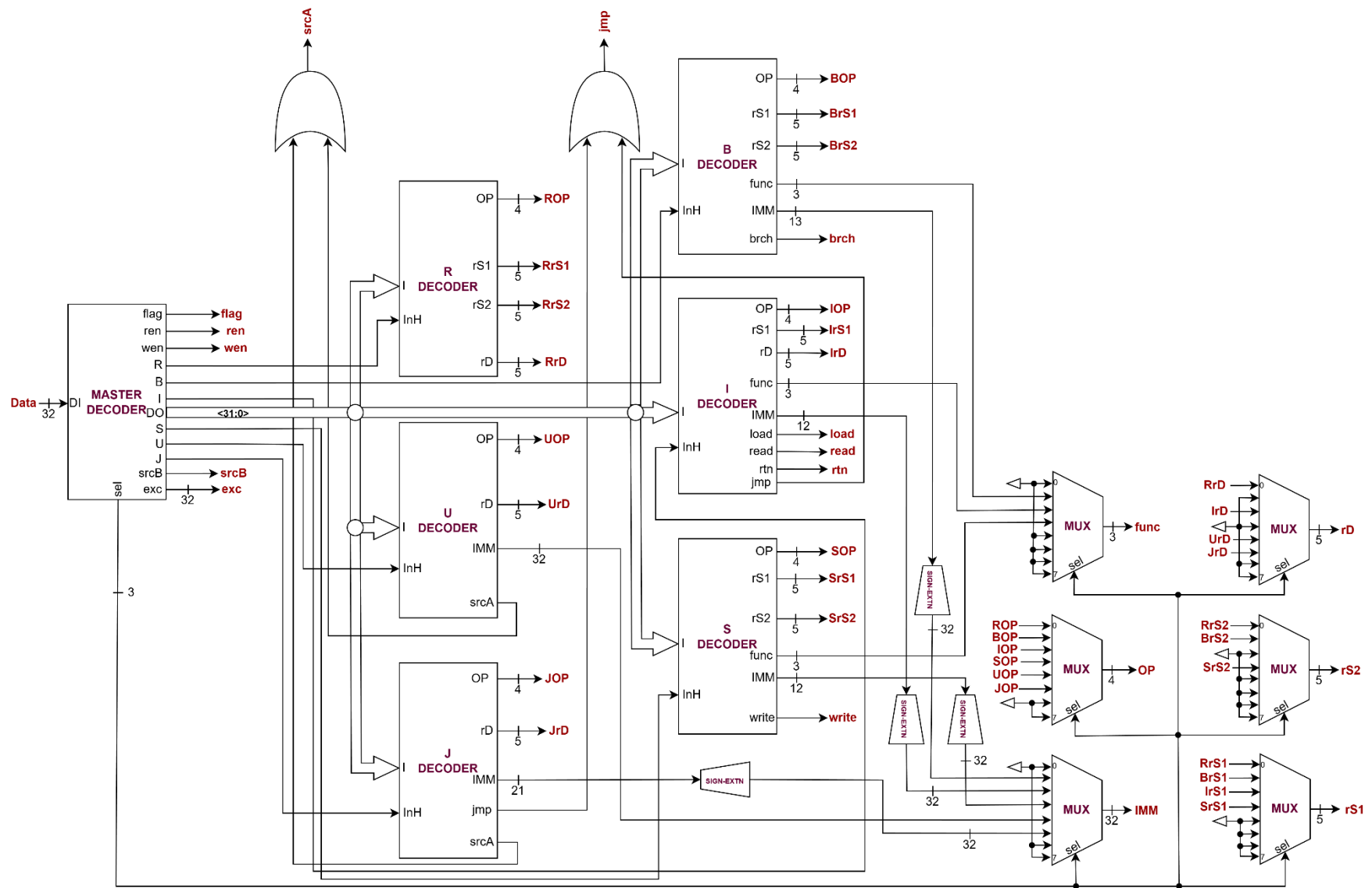


Figure 4: CODEC Architecture Datapath Design

3.2 GPR Architecture Design

A GPR is the fastest and most rudimentary form of memory in CPU architecture. Its primary function is to store the numerical result of an operation and retrieve them as operands. Zircon consists of a file of 32 registers each storing 32-bit words as in Figure 5. The architecture design is based on Figure 6. All registers are cleared when reset is asserted HIGH. RISC-V International (2026) standardises a read-only register x00 that stores only a null word (RISC-V International 2026). Whereas the remaining read-write registers may store words from the Data bus.

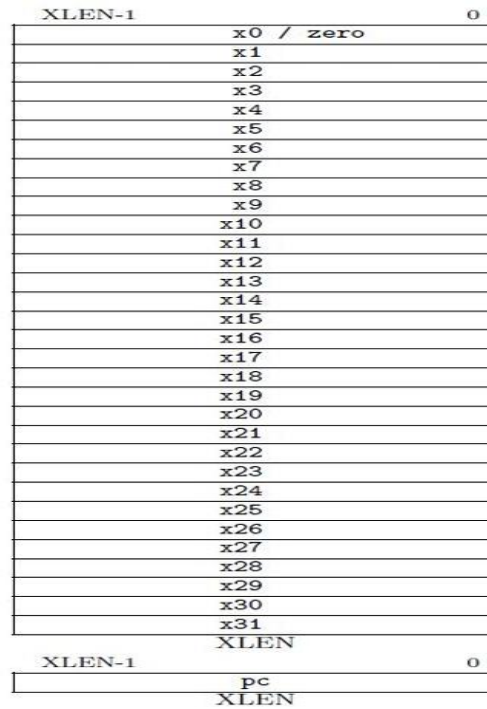


Figure 5: GPR and PC Structure (Source: RISC-V International 2026)

The file location of source registers is indexed at ports rS1 and rS2. An address LUT decodes rS1 and rS2 as addX and addY respectively. These perform MUX selections from the register file to their associated output ports. Operands are held for reading at their ports when read_enable (ren) is asserted HIGH. Destination register file locations are indexed at port rD. From this the address LUT asserts the associated register port HIGH. The selected register then stores a word from the Data bus when write_enable (wen) is asserted HIGH during a rising clock edge.

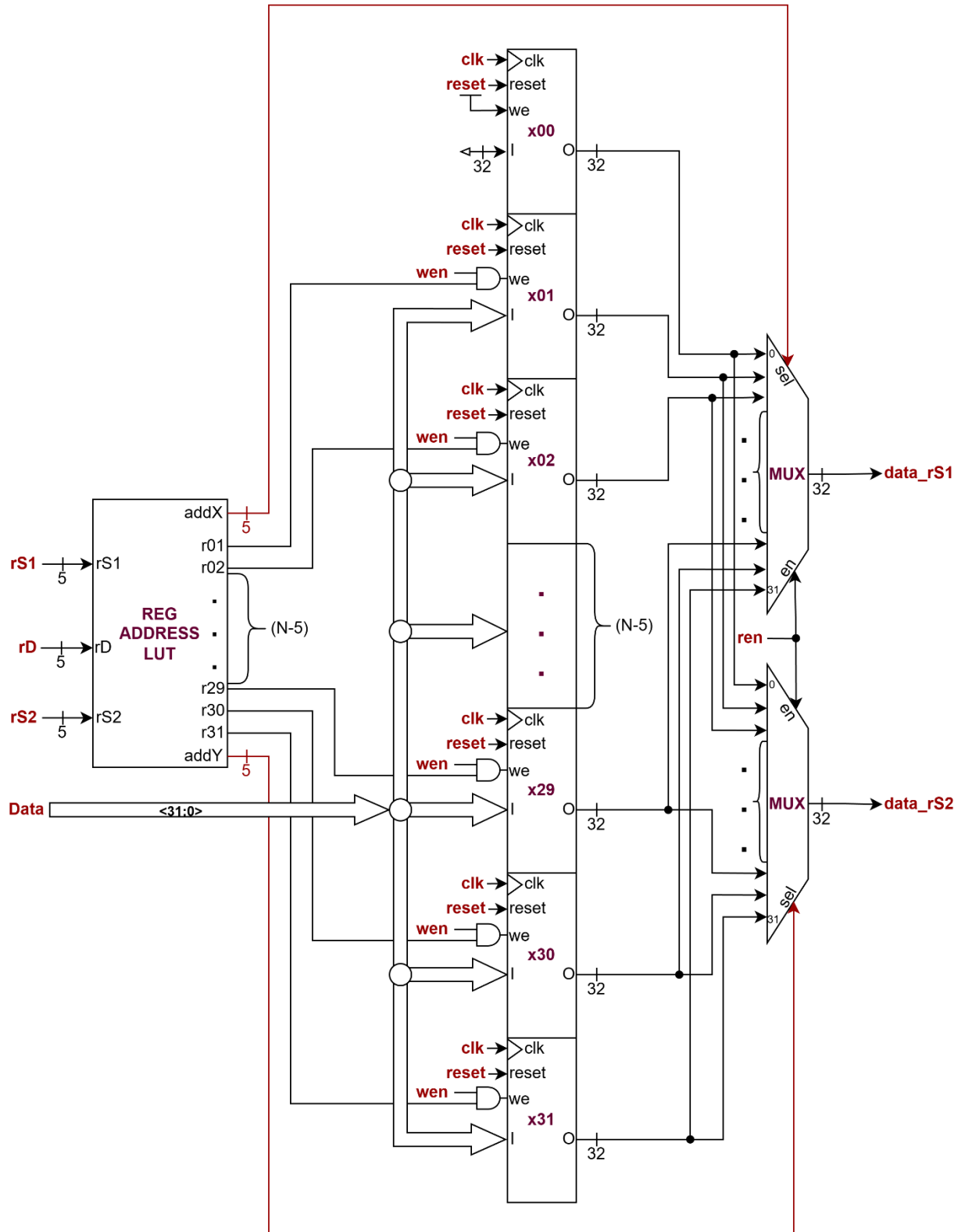


Figure 6: GPR File Architecture

3.3 ALU Architecture Design

Figure 7 illustrates the architecture design of the ALU. This block is responsible for executing all arithmetic and logical functions on operands A and B. It is capable of bitwise AND, OR, XOR and Shift logical operations. Arithmetic operations include Summation, Two's Complement Subtraction, (Un)Signed Comparison, and Shift. A carry flag (Cf) is raised only when a carry bit is pushed after arithmetic. A zero flag (Zf) is raised only when a logic or arithmetic result is zero. And an overflow flag (Of) is raised only when the carry-in to the MSB of an arithmetic result differs from its carry-out.

Table 3: ALU Control Logic

Function			Control Signal					
Mnemonic	Operation	OP	Carry	INV	SGN	LA	DIR	sel
AND	$R \rightarrow A \wedge B$	0000	0	0	0	0	0	000
OR	$R \rightarrow A \vee B$	0001	0	0	0	0	0	001
XOR	$R \rightarrow A \oplus B$	0010	0	0	0	0	0	010
ADD	$R \rightarrow A + B$	0011	0	0	0	0	0	011
SUB	$R \rightarrow A - B$	0100	1	1	0	0	0	011
SLT	$R \rightarrow \begin{cases} 1; A < B \\ 0; A \geq B \end{cases}$	0101	1	1	1	0	0	100
SLL	$R \rightarrow A \ll B$	0110	0	0	0	0	0	101
SRL	$R \rightarrow A \gg B$	0111	0	0	0	0	1	101
SRA	$R \rightarrow A \ggg B$	1000	0	0	0	1	1	101
-	$R \rightarrow A$	1010	0	0	0	0	0	110
SLTU	$R \rightarrow \begin{cases} 1; A < B \\ 0; A \geq B \end{cases}$	1100	1	1	0	0	0	100
-	$R \rightarrow B$	1111	0	0	0	0	0	111

All arithmetic except Shifts utilises a Ripple Adder. Both Subtractions and Unsigned Comparisons involve inversion of an operand and assertion of a carry bit for two's complement operation. The result of an Unsigned Comparison is determined by the carry flag and a null adder result. No carry and no zeroes results in only a set LSB since $|A| < |B|$. The result is otherwise cleared. Signed Comparisons obey the same logic when parity of sign-bits is detected. When no parity is detected, a predetermined set is output when $A < B$, or a clear if otherwise. Control of the ALU is governed by a LUT. This decodes a 4-bit operation code and implements the control logic given by Table 3.

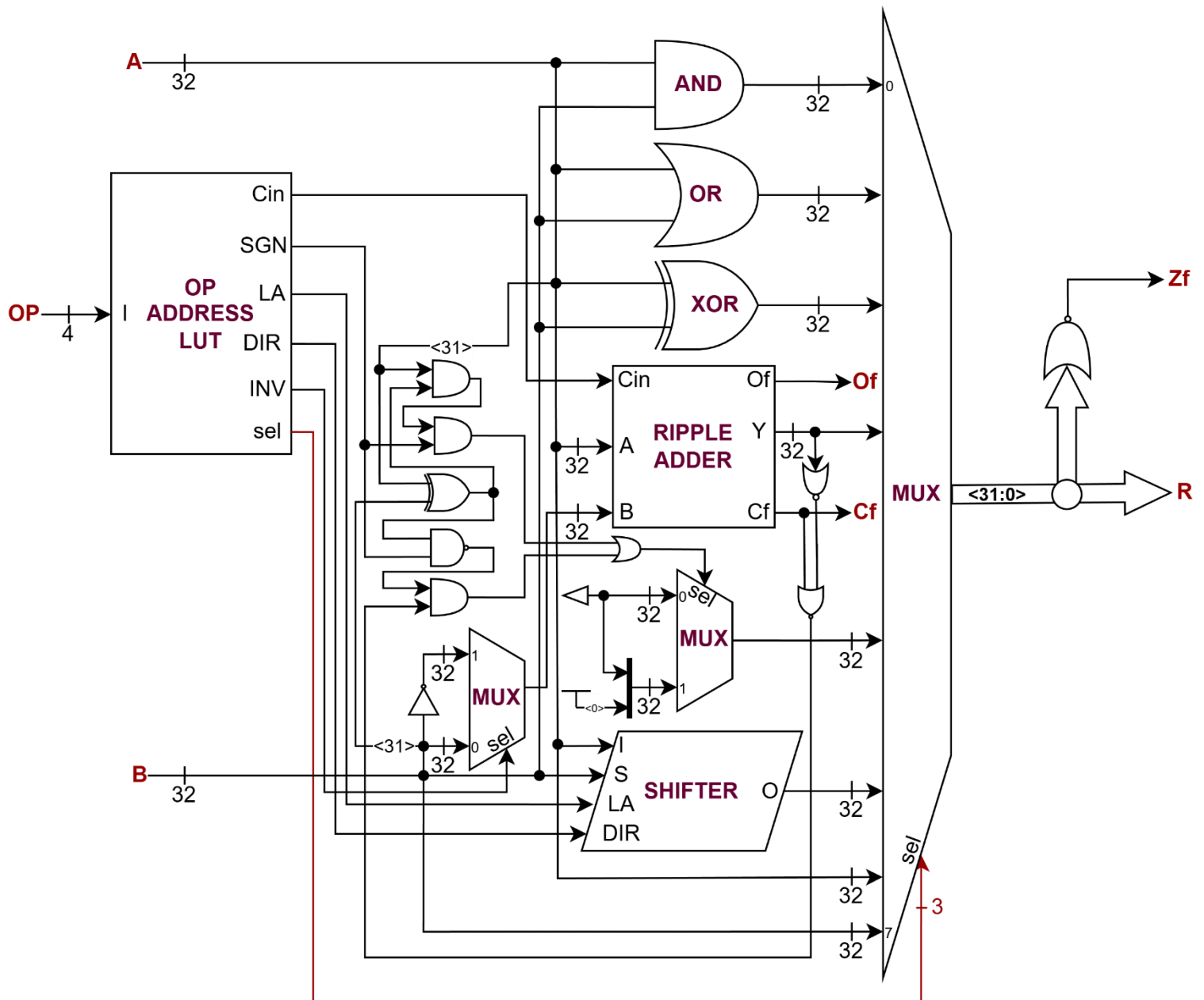


Figure 7: ALU Architecture

3.3.1 Ripple Adder Architecture Design

Design of the Ripple Adder module is based on the architecture of Figure 8. Each sub-module implements bitwise addition of a carry bit with operands A and B. The resulting sum is inserted onto bus Y. An N-bit word requires N sub-modules to implement full bitwise addition. Hence thirty-two sub-modules are utilised to sum two 32-bit words. These are cascaded by asserting the preceding carry-out onto the proceeding carry-in. Carry-in is initialised at port Cin and the final carry-out is asserted onto port Cout. Cout directly infers the status of Cf. Of is determined by comparing the parity of Cf with its preceding carry-in. It is enabled only when sign parity of both operands is detected during an addition, or otherwise during a subtraction.

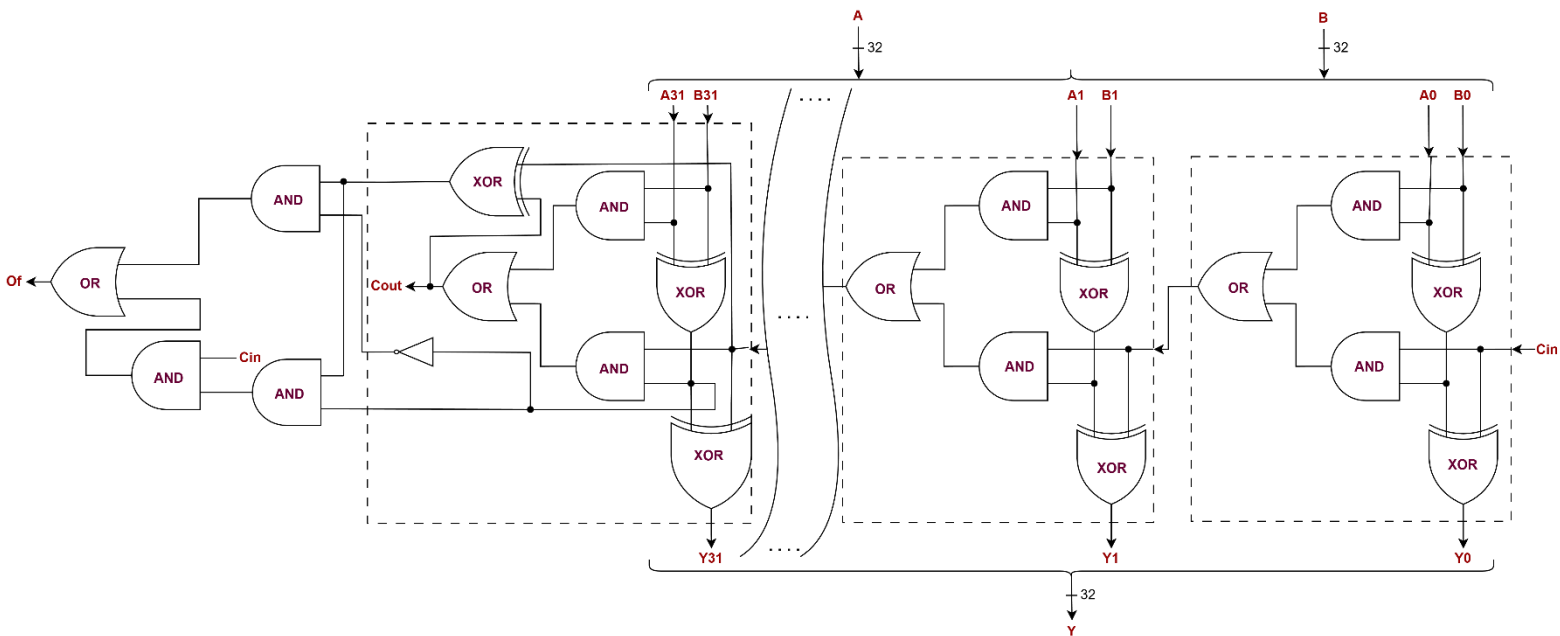


Figure 8: Ripple Adder Architecture

3.3.2 Shifter Architecture Design

Figure 9 illustrates the design of the Shifter architecture. It consists of a series of cascaded bitwise sub shifters. A sub shifter will shift a 32-bit word at port I by 1-bit in a direction determined by port DIR. Asserting DIR LOW results in a left shift. And raising DIR HIGH results in a right shift. The type of shift is controlled by port LA. Asserting LA LOW results in a logical shift. And raising LA HIGH results in an arithmetic shift.

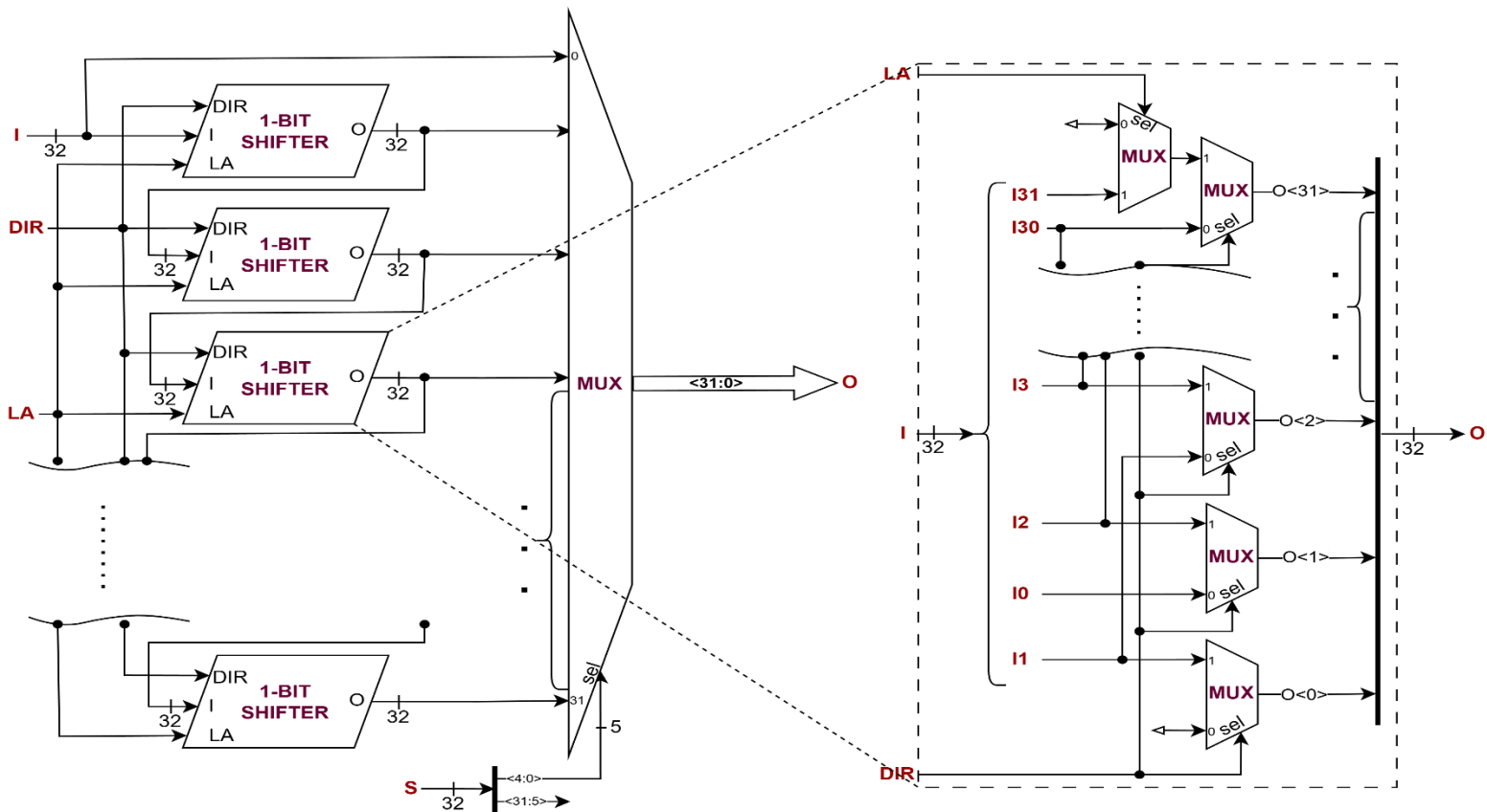


Figure 9: Shifter Architecture

The number of shifts on a word is proportional to the depth of cascaded sub shifters. This is selected through a MUX onto the bus O. There are exactly 2^5 possible shifts on a word. RISC-V International (2026) specifies that the number of shifts on an operand A is determined by the five LSBs of an operand B (RISC-V International 2026). Hence the five LSBs at port S determines MUX selections while the rest are discarded.

3.4 L1-D Cache Architecture Design

Zircon stores data and instructions in Level-1 cache, which is partitioned into L1-D and L1-I cache respectively. Design of L1-D follows the architecture of Figure 11 and primarily consists of a MAC and RAM. The RAM block is a byte addressable memory unit with a capacity of 1MB or 256-words. A word consists of 4 bytes. Hence four address lines controlled by dedicated enable ports are required for parallel access.

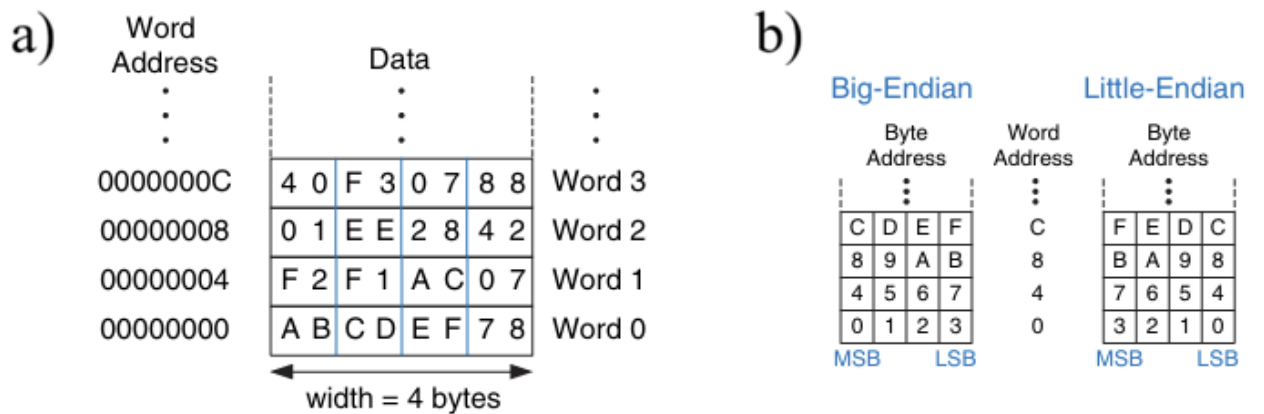


Figure 10: (a) Byte-Addressable Memory and (b) Big- and Little-Endian Addressing

(Source: Harris and Harris 2012)

Words on the DataI bus are partitioned into little-endian bytes as in Figure 10 (b). The address of each byte is assigned to an address line. Bytes are available for storage when their respective address lines are enabled. They are inhibited and padded by zeroes when otherwise. Bytes are stored in parallel when the write port is asserted HIGH at a rising clock edge. Words on the DataO bus consists of departitioned little-endian bytes. Bytes are available for loading when their respective address lines are enabled or inhibited when otherwise. Padding will then consist of zeroes when the sgnEN port is asserted LOW. Sign bits are otherwise padded when the sgnEN port is asserted HIGH. Bytes are loaded in parallel when the read port is asserted HIGH at a rising clock edge.

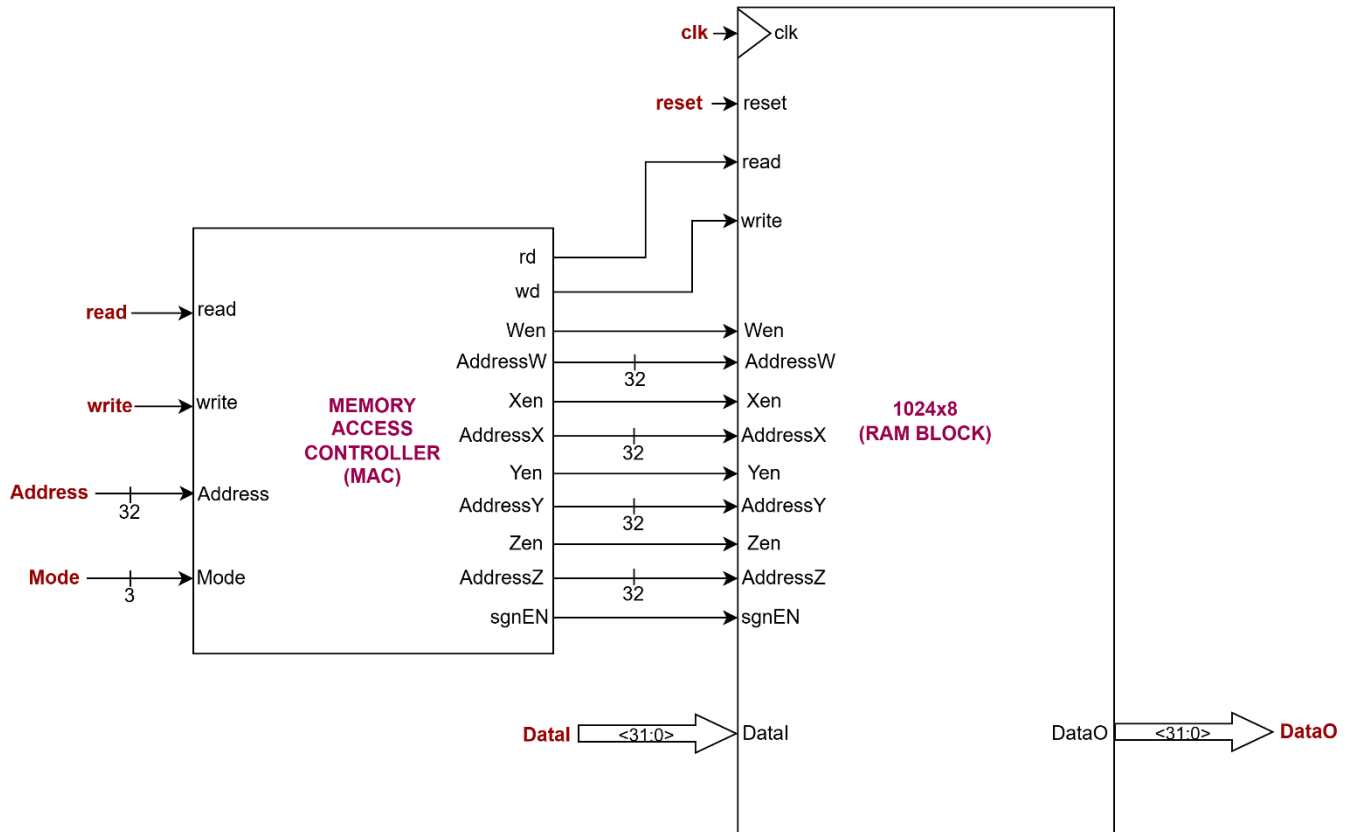


Figure 11: L1-D Cache Architecture

RAM access is coordinated by the MAC block which is only enabled during a read or write. The MAC first aligns a received address against a valid address width of four bytes. It then points to a valid memory location of a word. This infers the addresses of each byte to be written or read which ensures no overlap. The MAC will then assert either write_done (wd) or read_done (rd) HIGH which proceeds RAM operations. These will then self-reset at the next memory operation. A signed byte, signed half, or word may be stored or loaded at modes 0x0, 0x1, or 0x2 respectively. An unsigned byte or half may be loaded at modes 0x4 or 0x5.

4 System Test Plan

Version: v1.0.0

Date: 06/07/26

Introduction

This chapter describes the approach for verifying Zircon and ensures that it meets the specifications outlined in the System Requirements Specification (SRS). Test cases for each requirement will be analysed by black box and white box testing where relevant. A detailed analysis of how each component is validated by use of system testing is provided

Terms and Acronyms

Table 4: STP Abbreviations

Acronym	Definition
PC	Personal Computer
SUT	System Under Test
UUT	Unit(s) Under Test

Table 5: STP Technical Terms

Terms	Definition
Environmental Needs	These are all the necessary software and hardware tools needed when conducting the test cases.

Test Cases

This section provides a description of the test cases for the requirements mentioned in the SRS chapter. These test cases are supplemented with desired results, environmental needs, and post-test results.

4.1 SRS REQ3-1

Table 6: SRS REQ3-1 Description

Requirement Identifier	SRS Description
REQ3-1	RV32I ISA Support
<i>REQ3-1.1</i>	<i>Read</i>
<i>REQ3-1.2</i>	<i>Write</i>
<i>REQ3-1.3</i>	<i>Load</i>
<i>REQ3-1.4</i>	<i>Store</i>

Purpose of Test

To ensure that the system complies with the standards RV32I ISA. To ensure that variables can be written to and read from the system. And that variables can be loaded from and stored to memory.

Test Specification Handler

The unique identifier assigned to this test case is “TC3-1x”.

Test Item

UUT are the CODEC, GPR, and L1-D Cache. The SUT is the Zircon CPU. This test case is done by first inputting all base instructions of the RV32I ISA and then validating their behaviour against Table B-1. Literals will be written to and then read from the GPR. Literals will then be stored to and then loaded from L1-D cache.

Test Input

Table 7: Test Case (TC3-1) Inputs

Test	Test Case	Input
TC3-1	RV32I Compliance	Table B-1
TC3-1.2	Write	ADDI x1,x0,0xFF ORI x3,x1,0xFFFF
TC3-1.1	Read	ADD x1,x0,x2
TC3-1.4	Store	SB x3,x0,0x0
		SH x3,x0,0x1
		SW x3,x0,0x2
TC3-1.3	Load	LB x4,x0,0x0
		LH x5,x0,0x1
		LW x6,x0,0x2

Desired Output

Table 8: Test Case (TC3-1) Desired Outputs

Test	Test Case	Desired Output
TC3-1	RV32I Compliance	Table B-1
TC3-1.2	Write	$x1 \rightarrow 0xFF$ $x3 \rightarrow 0xFFFFFFFF$
TC3-1.1	Read	$x2 \rightarrow 0xFF$
TC3-1.4	Store	$Mem[0] \leftarrow 0xFF$
		$Mem[1] \leftarrow 0xFFFF$
		$Mem[2] \leftarrow 0xFFFFFFFF$
TC3-1.3	Load	$x4 \rightarrow 0xFF$
		$x5 \rightarrow 0xFFFF$
		$x6 \rightarrow 0xFFFFFFFF$

Environmental Needs

- Hardware:
 - PC
- Software:
 - AMD Vivado Design Suite 2022.2
 - XSim; and
 - ZASM v1.0.0 (Appendix C – Zircon Assembler v1.0.0).

4.2 SRS REQ3-2

Table 9: SRS REQ3-2 Description

Requirement Identifier	SRS Description
REQ3-2	Basic Logic
<i>REQ3-2.1</i>	<i>Bitwise OR</i>
<i>REQ3-2.2</i>	<i>Bitwise XOR</i>
<i>REQ3-2.3</i>	<i>Bitwise AND</i>
<i>REQ3-2.4</i>	<i>Bitwise NOT</i>
<i>REQ3-2.5</i>	<i>Left Shift</i>
<i>REQ3-2.6</i>	<i>Right Shift</i>

Purpose of Test

To ensure that the system can execute basic logic operations between two operands and output the correct result.

Test Specification Handler

The unique identifier assigned to this test case is “TC3-2x”.

Test Item

The UUT are the CODEC, GPR, and ALU. The SUT is the Zircon CPU. This test case is done by applying a logic function on two registers, or on a register and a literal, then confirming that the output is mathematically correct.

Test Input

Table 10: Test Case (TC3-2) Inputs

Test	Test Case	Input
TC3-2.1	Bitwise OR	ORI x7,x0,0xA
		OR x8,x4,x7
TC3-2.2	Bitwise XOR	XORI x9,x7,0xB
		XOR x10,x8,x7
TC3-2.3	Bitwise AND	ANDI x11,x0,0xC
		AND x12,x8,x7
TC3-2.4	Bitwise NOT	XORI x9,x6,0xFF
		XOR x5,x5,x5
TC3-2.5	Left Shift	SLLI x13,x3,0x4
		SLL x14,x3,x7
TC3-2.6	Right Shift	SRLI x15,x7,0x2
		SRL x16,x3,x0

Desired Output

Table 11: Test Case (TC3-2) Desired Outputs

Test	Test Case	Desired Output
TC3-2.1	Bitwise OR	$x7 \rightarrow 0xA$
		$x8 \rightarrow 0xFF$
TC3-2.2	Bitwise XOR	$x9 \rightarrow 0x1$
		$x10 \rightarrow F5$
TC3-2.3	Bitwise AND	$x11 \rightarrow 0$
		$x12 \rightarrow 0xA$
TC3-2.4	Bitwise NOT	$x9 \rightarrow 0$
		$x5 \rightarrow 0$
TC3-2.5	Left Shift	$x13 \rightarrow 0xFFFFFFFF0$
		$x14 \rightarrow 0xFFFFFC00$
TC3-2.6	Right Shift	$x15 \rightarrow 0x2$
		$x16 \rightarrow 0xFFFFFFFF$

Environmental Needs

- Hardware:
 - PC
- Software:
 - AMD Vivado Design Suite 2022.2;
 - XSim; and
 - ZASM v1.0.0 (Appendix C – Zircon Assembler v1.0.0).

4.3 SRS REQ3-3

Table 12: SRS REQ3-3 Description

Requirement Identifier	SRS Description
REQ3-3	Basic Arithmetic
<i>REQ3-3.1</i>	<i>Add</i>
<i>REQ3-3.2</i>	<i>Subtract</i>
<i>REQ3-3.3</i>	<i>Compare</i>
<i>REQ3-3.4</i>	<i>Right Shift</i>

Purpose of Test

To ensure that the system can execute basic arithmetic operations between two operands and output the correct result.

Test Specification Handler

The unique identifier assigned to this test case is “TC3-3x”.

Test Item

The UUT are the CODEC, GPR, and ALU. The SUT is the Zircon CPU. This test case is done by applying an arithmetic function on two registers, or on a register and a literal, then confirming that the output is mathematically correct.

Test Input

Table 13: Test Case (TC3-3) Inputs

Test	Test Case	Input
TC3-3.1	Additon	ADDI x16,x0,0x7
		ADD x17,x10,x7
TC3-3.2	Subtraction	SUB x18,x16,x17
		SUB x23,x17,x16
TC3-3.3	Comparison	SLTI x19,x16,0x0
		SLT x20,x16,x18
TC3-3.4	Right Shift	SRAI x21,x23,0x3
		SRA x22,x23,x7

Desired Output

Table 14: Test Case (TC3-3) Desired Outputs

Test	Test Case	Desired Output
TC3-3.1	Addition	$x16 \rightarrow 0x7$
		$x17 \rightarrow 0xFF$
TC3-3.2	Subtraction	$x18 \rightarrow 0xFFFFFFFF08$ $Cf = 0$
		$x23 \rightarrow 0xF8$ $Cf = 1$
TC3-3.3	Comparison	$x19 \rightarrow 0x0$ $Cf = 1; Zf = 1$
		$x20 \rightarrow 0x0$ $Cf = 0; Zf = 1$
TC3-3.4	Right Shift	$x21 \rightarrow 0x1F$
		$x22 \rightarrow 0x0$

Environmental Needs

- Hardware:
 - PC
- Software:
 - AMD Vivado Design Suite 2022.2;
 - XSim; and
 - ZASM v1.0.0 (Appendix C – Zircon Assembler v1.0.0).

4.4 SRS REQ3-4, REQ3-5, and REQ3-6

Table 15: SRS REQ3-4, -5, -6 Description

Requirement Identifier	SRS Description
REQ3-4	Branch Conditional
REQ3-5	Jump
REQ3-6	No Operation (NOP)

Purpose of Test

To ensure that the system can branch off from an instruction when conditions exist.

To ensure that the system can jump from and then return to an instruction. And to ensure that the system can perform a NOP pseudo-instruction.

Test Specification Handler

The unique identifier assigned to this test case is “TC3-xx”.

Test Item

The UUT are the PC, ROM, and CODEC. The SUT is the Zircon CPU. This test case is done by applying Branch and Jump instructions on two registers, or on a register and a literal, and the following instructions to be executed will then be verified. NOPs will be applied and verified with recommended base instruction combinations of (RISC-V International 2026).

Test Input

Table 16: Test Case (TC3-4) Inputs

Test	Test Case	Input
TC3-4	Branch Conditional	BEQ x8,x5,0x8 ADDI x5,x5,0x1 NOP BEQ x1,x2,0x8 ADDI x9,x9,0x1 NOP
		BNE x8,x7,0x8 ADDI x11,x11,0x0 NOP BNE x8,x4,0x8 ADDI x11,x11,0x1 NOP
		BLT x8,x9,0x8 ADDI x9,x9,0xFFFF NOP BLT x8,x9,0x8 XOR x9,x9,x9 NOP
		BGE x8,x9,0x8 ADDI x5,x5,0xC NOP BGE x5,x7,0x8 XOR x5,x5,x5 NOP

Table 17: Test Case (TC3-5) Inputs

Test	Test Case	Input
TC3-5	Jump	JAL x1,0x8 ADDI x5,x5,0xB ADDI x9,x9,0xE JALR x0,x1,0xC NOP

Table 18: Test Case (TC3-6) Inputs

Test	Test Case	Input
TC3-6	No Operation (NOP)	ADDI x0,x0,0x0

Desired Output

Table 19: Test Case (TC3-4) Desired Outputs

Test	Test Case	Desired Output
TC3-4	Branch Conditional	$x5 \rightarrow 0x1$ $x9 \rightarrow 0x0$
		$x11 \rightarrow 0x0$ $x11 \rightarrow 0x1$
		$x9 \rightarrow 0xFFFFFFFF$ $x9 \rightarrow 0x0$
		$x5 \rightarrow 0x0$

Table 20: Test Case (TC3-5) Desired Outputs

Test	Test Case	Desired Output
TC3-5	Jump	$x5 \rightarrow 0x0$ $x9 \rightarrow 0xE$

Table 21: Test Case (TC3-6) Desired Outputs

Test	Test Case	Desired Output
TC3-6	No Operation (NOP)	$x0 \rightarrow 0x0$

Environmental Needs

- Hardware:
 - PC
- Software:
 - AMD Vivado Design Suite 2022.2;
 - XSim; and
 - ZASM v1.0.0 (Appendix C – Zircon Assembler v1.0.0).

5 Results

Figure 12 and Figure 13 illustrate the hardware and power utilisation of the synthesised Zircon v1.0.0 CPU core. Figure 14 through Figure 18 gives the results of the System Test Plan (STP) and is summarised by Table 22. System requirements are finally traced to the STP by Table 23 and functionally monitored by Table 24.

Resource	Utilization	Available	Utilization (%)
LUT	15359	47200	32.54
FF	9249	94400	9.80
IO	35	210	16.67
BUFG	4	32	12.50

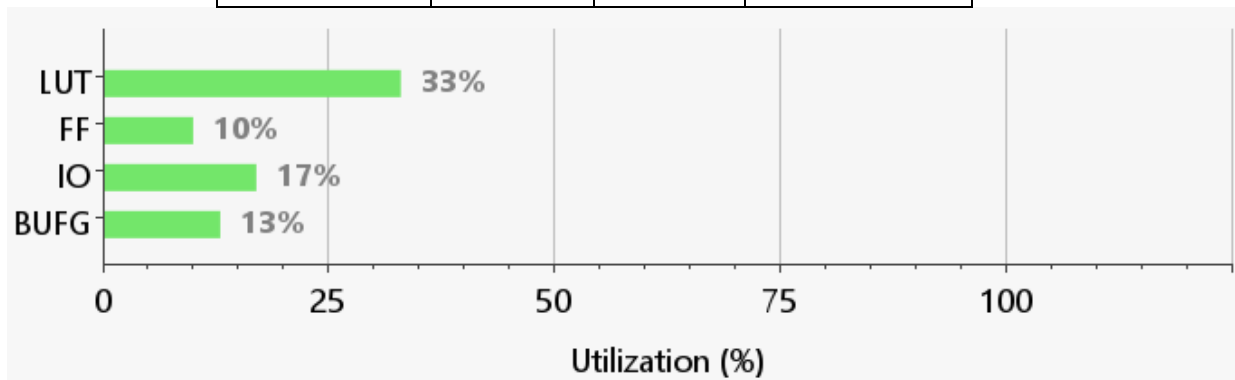


Figure 12: Hardware Utilisation

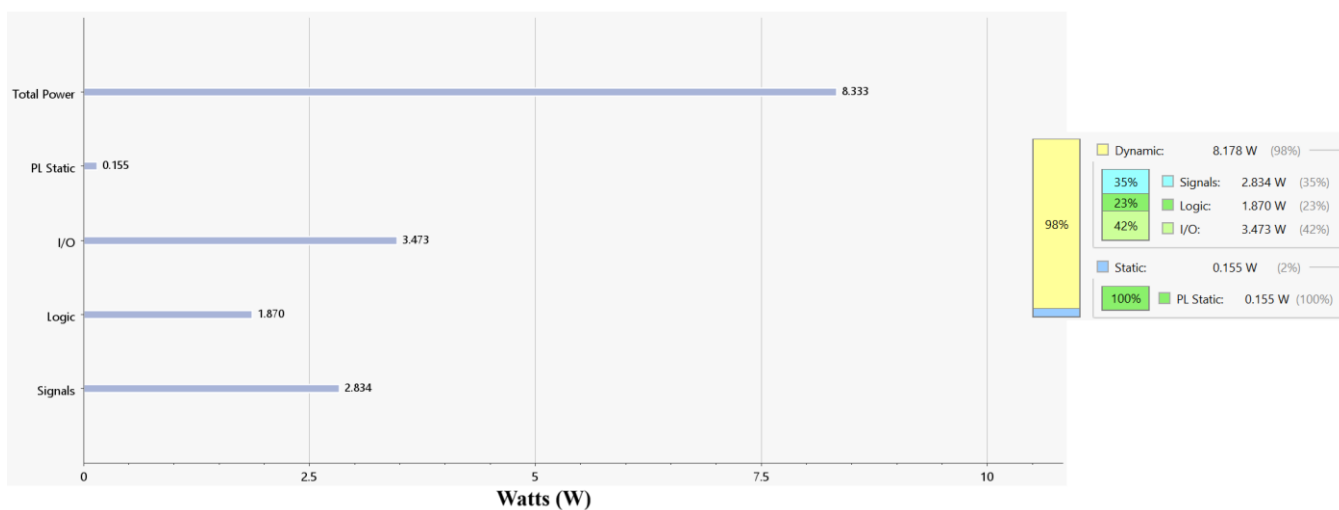


Figure 13: Power Utilisation

5.1 Post System Test Plan

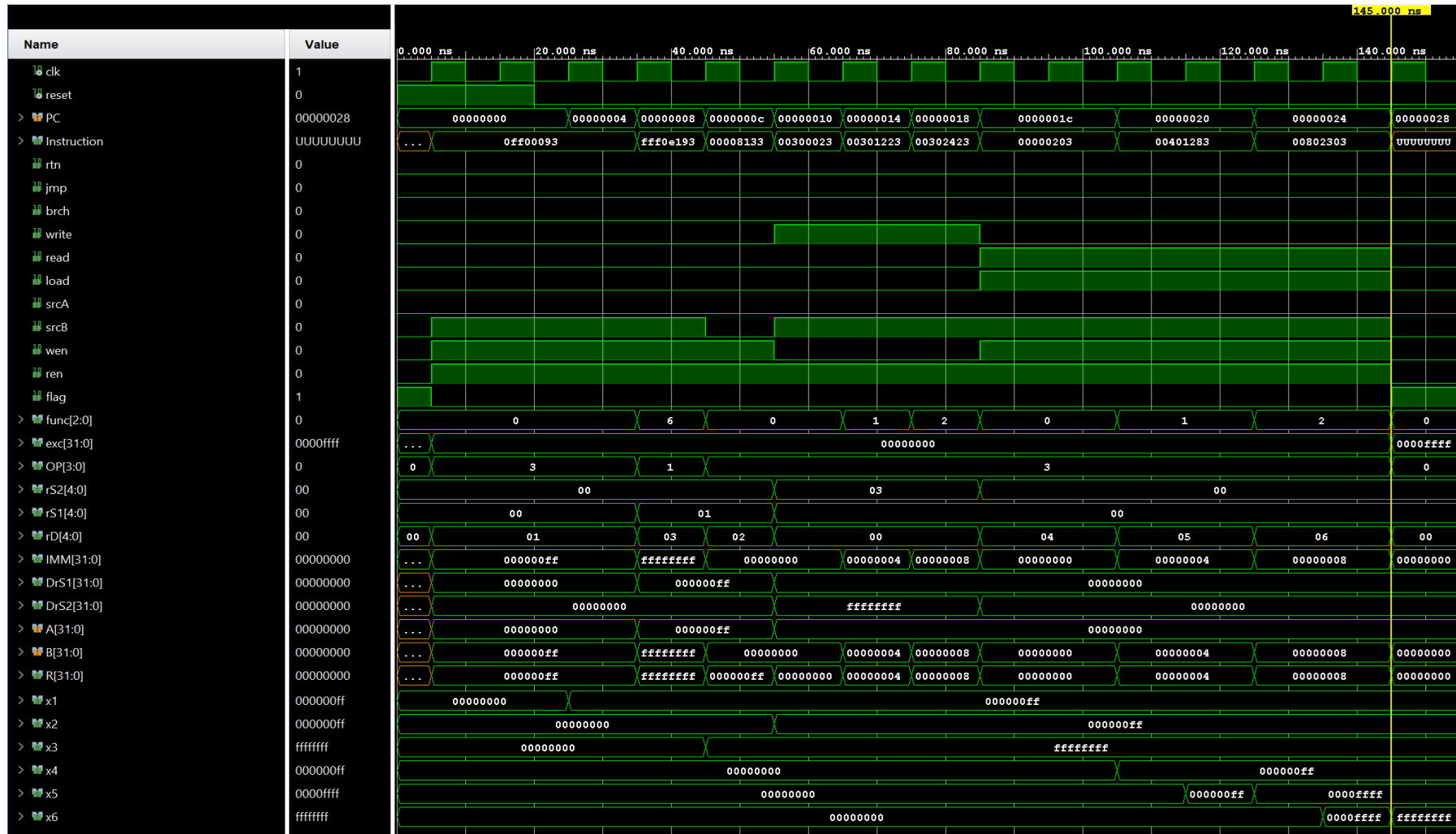


Figure 14: (TC3-1) Reads, Writes, Loads, and Stores

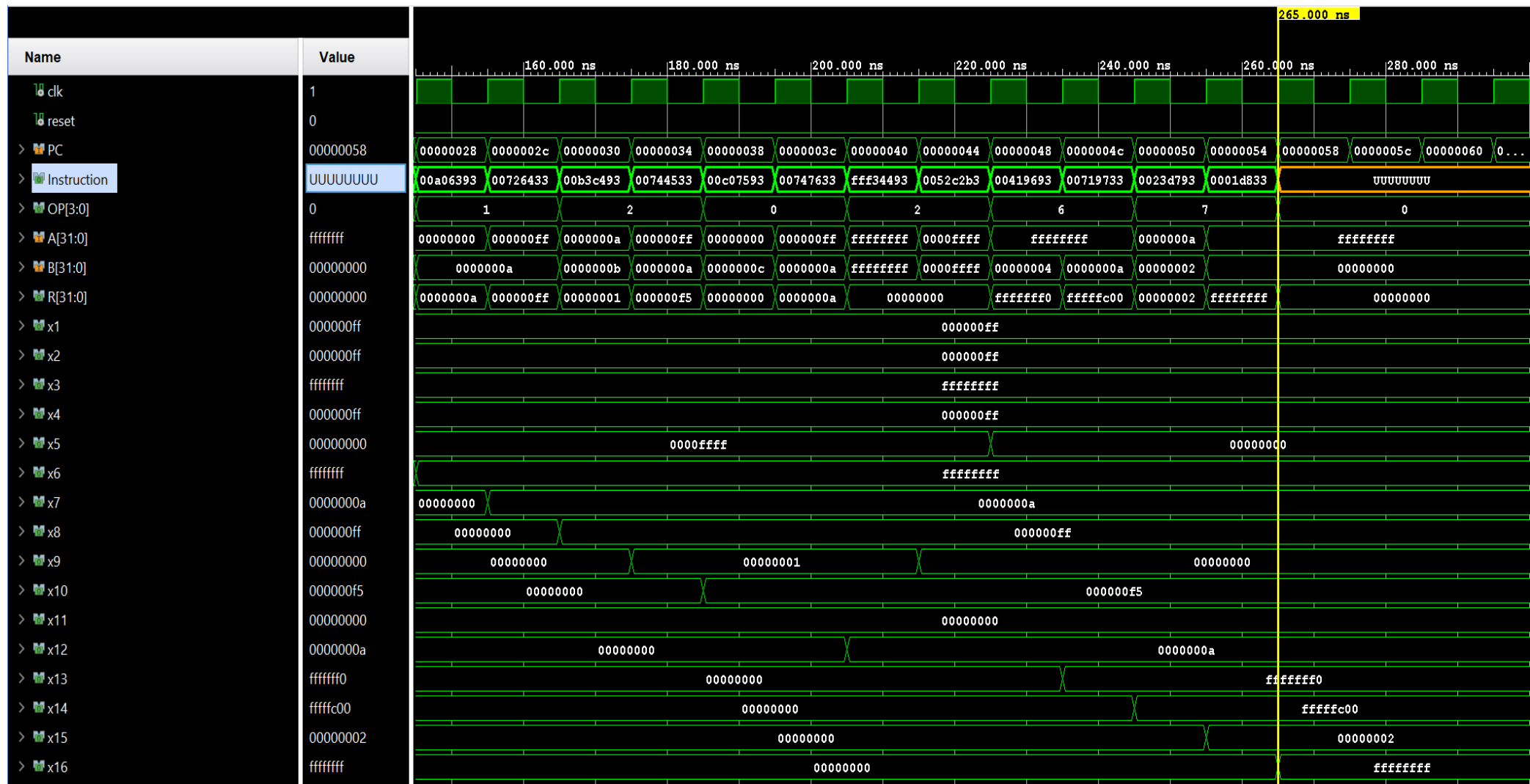


Figure 15: (TC3-2) Basic Logic

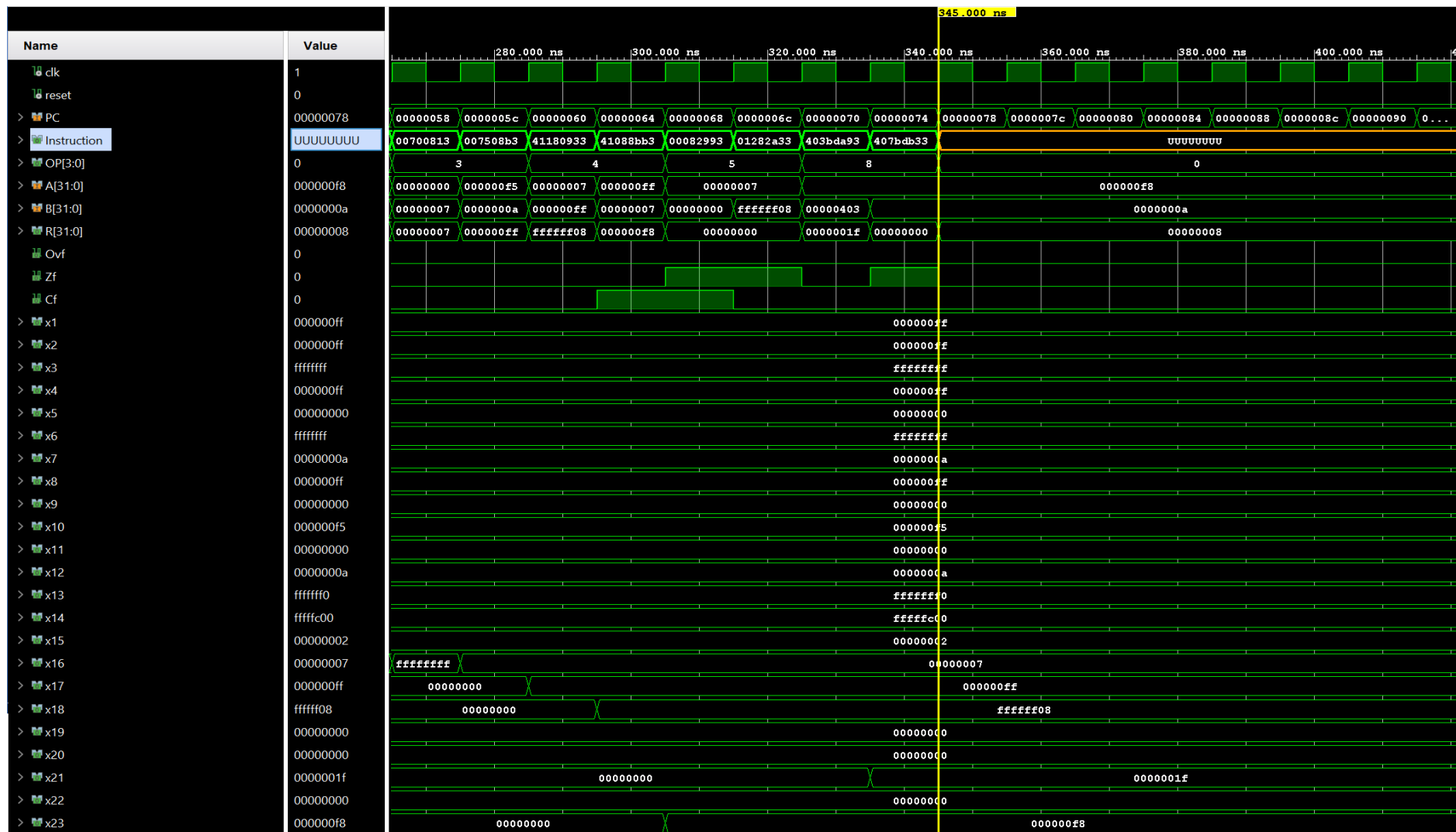
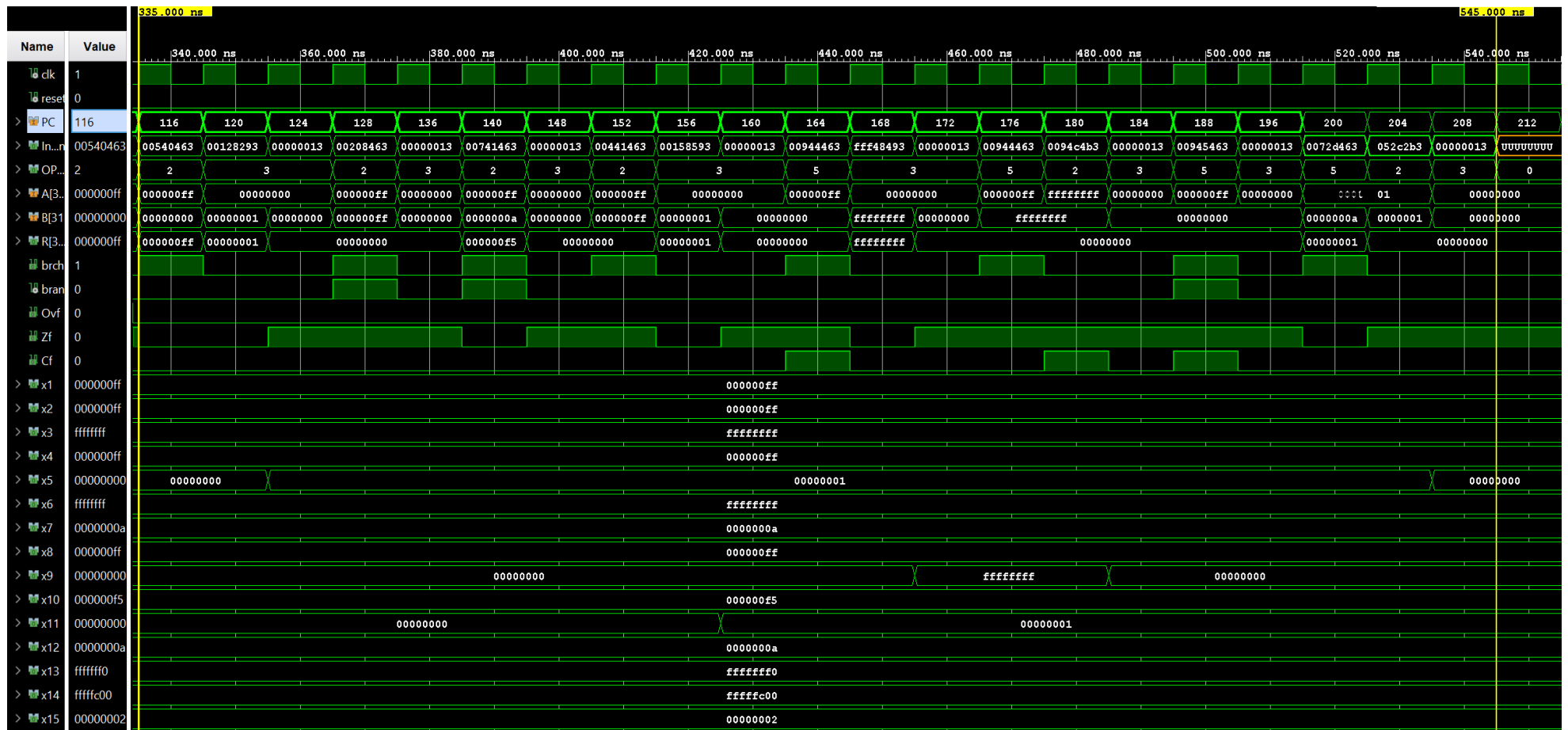


Figure 16: (TC3-3) Basic Arithmetic



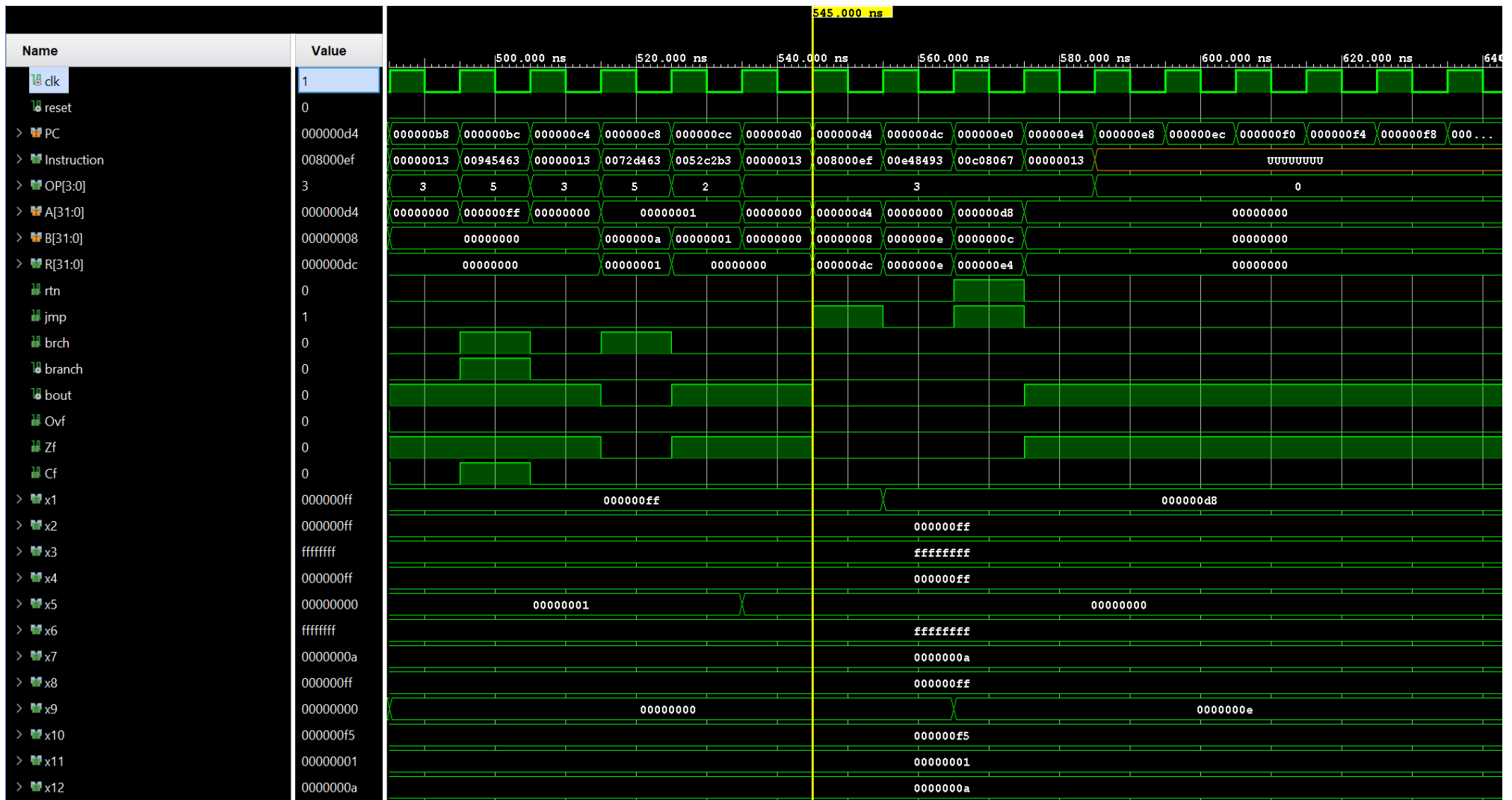


Figure 18: (TC3-5/6) Call, Return, and NOP

5.2 System Test Plan Summary

Table 22 summarises the test cases from the System Test Plan.

Table 22: System Test Plan Summary

Test Case Handler	Implemented (Y/N)	Pass Criteria	Pass/Fail/(NA)
TC3-1	Y	Correct instruction identification and control logic assertion per Table B-1	Pass
TC3-1.2	Y	<i>Correct literals written to correct destination regs.</i>	<i>Pass</i>
TC3-1.1	Y	<i>Contents of written regs are read to correct destination reg.</i>	<i>Pass</i>
TC3-1.4	Y	<i>Byte, half, and word of a literal are cached at the correct memory addresses.</i>	<i>Pass</i>
TC3-1.3	Y	<i>Byte, half, and word are loaded from correct cache addresses to correct destination regs.</i>	<i>Pass</i>
TC3-2	Y	Produces mathematically correct result of supported logic.	Pass
TC3-2.1	Y	<i>Correct bitwise OR of regs and literals, result written to correct destination.</i>	<i>Pass</i>
TC3-2.2	Y	<i>Correct bitwise XOR of regs and literals, result written to correct destination.</i>	<i>Pass</i>
TC3-2.3	Y	<i>Correct bitwise AND of regs and literals, result written to correct destination.</i>	<i>Pass</i>
TC3-2.4	Y	<i>Correct bitwise inversion of regs and literals, result written to correct destination.</i>	<i>Pass</i>
TC3-2.5	Y	<i>Shifts a reg to the left by a literal or reg, padding vacancies with zeroes. Correct result written to correct reg.</i>	<i>Pass</i>
TC3-2.6	Y	<i>Shifts a reg to the right by a literal or reg, padding vacancies with zeroes. Correct result written to correct reg.</i>	<i>Pass</i>
TC3-3	Y	Produces mathematically correct result of supported arithmetic.	Pass
TC3-3.1	Y	<i>Correct addition of regs and literals, result written to correct reg.</i>	<i>Pass</i>
TC3-3.2	Y	<i>Correct subtraction of regs, result written to correct reg.</i>	<i>Pass</i>
TC3-3.3	Y	<i>Correct comparison of regs and literals, result written to correct destination.</i>	<i>Pass</i>
TC3-3.4	Y	<i>Shifts a reg to the right by a literal or reg, padding vacancies with sign-bit. Correct result written to correct reg.</i>	<i>Pass</i>
TC3-4	Y	Correctly compares regs then loads correct target address to PC when conditions apply.	Pass
TC3-5	Y	Writes current PC address to correct reg and loads new address onto PC. Correctly reads stored PC address from reg then loads onto PC with offset.	Pass
TC3-6	Y	Summation of null literal and null reg. Null result written to null reg.	Pass

5.3 Requirements Traceability Matrix

The requirements traceability matrix in Table 23 is based on the –

- System Requirements Specification v1.0.0;
- System Architecture Design v1.0.0; and
- System Test Plan v1.0.0.

Table 23: Requirements Traceability Matrix

System Requirements Specification (SRS)			System Test Plan (STP)
Identifier	Functional?	Priority	Identifier
REQ1-1	Yes	Necessary	TC3-x.x
REQ2-1	No	Low	-
REQ3-1	Yes	Necessary	TC3-1
<i>REQ3-1.1</i>	Yes	Necessary	TC3-1.1
<i>REQ3-1.2</i>	Yes	Necessary	TC3-1.2
<i>REQ3-1.3</i>	Yes	Necessary	TC3-1.3
<i>REQ3-1.4</i>	Yes	Necessary	TC3-1.4
REQ3-2	Yes	Necessary	TC3-2
<i>REQ3-2.1</i>	Yes	Necessary	TC3-2.1
<i>REQ3-2.2</i>	Yes	Necessary	TC3-2.2
<i>REQ3-2.3</i>	Yes	Necessary	TC3-2.3
<i>REQ3-2.4</i>	Yes	Necessary	TC3-2.4
<i>REQ3-2.5</i>	Yes	Necessary	TC3-2.5
<i>REQ3-2.6</i>	Yes	Necessary	TC3-2.6
REQ3-3	Yes	Necessary	TC3-3
<i>REQ3-3.1</i>	Yes	Necessary	TC3-3.1
<i>REQ3-3.2</i>	Yes	Necessary	TC3-3.2
<i>REQ3-3.3</i>	Yes	Necessary	TC3-3.3
<i>REQ3-3.4</i>	Yes	Necessary	TC3-3.4
REQ3-4	Yes	Necessary	TC3-4
REQ3-5	Yes	Necessary	TC3-5
REQ3-6	Yes	Necessary	TC3-6

Table 24 monitors the status of each system requirement.

Table 24: Requirements Status Monitoring

Legend:

Completed	
In-progress	
Not Completed	

Requirement	Specification	Design	Implementation	Testing
REQ1-1				
REQ2-1				
REQ3-1				
<i>REQ3-1.1</i>				
<i>REQ3-1.2</i>				
<i>REQ3-1.3</i>				
<i>REQ3-1.4</i>				
REQ3-2				
<i>REQ3-2.1</i>				
<i>REQ3-2.2</i>				
<i>REQ3-2.3</i>				
<i>REQ3-2.4</i>				
<i>REQ3-2.5</i>				
<i>REQ3-2.6</i>				
REQ3-3				
<i>REQ3-3.1</i>				
<i>REQ3-3.2</i>				
<i>REQ3-3.3</i>				
<i>REQ3-3.4</i>				
REQ3-4				
REQ3-5				
REQ3-6				

6 Conclusion

A 32-bit dual cycle, single core, in-order CPU was designed on the unprivileged RV32I ISA to (RISC-V International 2026) standards. Testing on a 100MHz clock verified that the design is capable of reads, writes, loads and stores. It executes basic logic and arithmetic, conditionals, calls, returns and NOPs. And illegal instructions raise an exception code and flag once detected. The design completes fetch, decode, execute and memory stores in a single clock cycle. Whereas memory loads require dual cycle operation to account for RAM latency. Future works include a 2-stage pipeline to increase instruction throughput, and a trap handler to support interrupts.

References

- Harris, David Money, and Sarah L. Harris. 2012. *Digital Design and Computer Architecture*. 2nd. Edited by Todd Green and Nathaniel McFadden. Elsevier India. Accessed June 07, 2026.
- RISC-V International. 2026. “Ratified Specification: RISC-V.” Vers. 20260120. *RISC-V*. 20 January. Accessed May 18, 2026. <https://riscv.org/specifications/ratified/>.

Appendix A – Extracts of RISC-V Unprivileged ISA

31	27	26	25	24	20	19	15	14	12	11		7	6	0	
funct7				rs2		rs1		funct3		rd			opcode		R-type
imm[11:0]						rs1		funct3		rd			opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]			opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]			opcode		B-type
imm[31:12]										rd			opcode		U-type
imm[20 10:1 11 19:12]										rd			opcode		J-type

RV32I Base Instruction Set							
imm[31:12]				rd		0110111	LUI
imm[31:12]				rd		0010111	AUIPC
imm[20 10:1 11 19:12]				rd		1101111	JAL
imm[11:0]			rs1	000	rd		1100111 JALR
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]		1100011	BEQ
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]		1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]		1100011	BLT
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]		1100011	BGE
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]		1100011	BLTU
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]		1100011	BGEU
imm[11:0]			rs1	000	rd		0000011 LB
imm[11:0]			rs1	001	rd		0000011 LH
imm[11:0]			rs1	010	rd		0000011 LW
imm[11:0]			rs1	100	rd		0000011 LBU
imm[11:0]			rs1	101	rd		0000011 LHU
imm[11:5]	rs2	rs1	000	imm[4:0]		0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]		0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]		0100011	SW
imm[11:0]			rs1	000	rd		0010011 ADDI
imm[11:0]			rs1	010	rd		0010011 SLTI
imm[11:0]			rs1	011	rd		0010011 SLTIU
imm[11:0]			rs1	100	rd		0010011 XORI
imm[11:0]			rs1	110	rd		0010011 ORI

Appendix A – (Continued)

imm[11:0]			rs1	111	rd	0010011	ANDI
0000000	shamt		rs1	001	rd	0010011	SLLI
0000000	shamt		rs1	101	rd	0010011	SRLI
0100000	shamt		rs1	101	rd	0010011	SRAI
0000000	rs2		rs1	000	rd	0110011	ADD
0100000	rs2		rs1	000	rd	0110011	SUB
0000000	rs2		rs1	001	rd	0110011	SLL
0000000	rs2		rs1	010	rd	0110011	SLT
0000000	rs2		rs1	011	rd	0110011	SLTU
0000000	rs2		rs1	100	rd	0110011	XOR
0000000	rs2		rs1	101	rd	0110011	SRL
0100000	rs2		rs1	101	rd	0110011	SRA
0000000	rs2		rs1	110	rd	0110011	OR
0000000	rs2		rs1	111	rd	0110011	AND
fm	pred	succ	rs1	000	rd	0001111	FENCE
1000	0011	0011	00000	000	00000	0001111	FENCE.TSO
0000	0001	0000	00000	000	00000	0001111	PAUSE
000000000000			00000	000	00000	1110011	ECALL
000000000001			00000	000	00000	1110011	EBREAK

Appendix B – Control Matrix

Table B-1: Control Matrix

Instruction			Control Logic											
Mnemonic	Type	func7	func	ren	wen	rtn	jmp	brch	read	write	load	srcA	srcB	OP
ADD	R	0000000	000	1	1	0	0	0	0	0	0	0	0	0011
SUB		0100000	000	1	1	0	0	0	0	0	0	0	0	0100
SLL		0000000	001	1	1	0	0	0	0	0	0	0	0	0110
SLT		0000000	010	1	1	0	0	0	0	0	0	0	0	0101
SLTU		0000000	011	1	1	0	0	0	0	0	0	0	0	1100
XOR		0000000	100	1	1	0	0	0	0	0	0	0	0	0010
SRL		0000000	101	1	1	0	0	0	0	0	0	0	0	0111
SRA		0100000	101	1	1	0	0	0	0	0	0	0	0	1000
OR		0000000	110	1	1	0	0	0	0	0	0	0	0	0001
AND		0000000	111	1	1	0	0	0	0	0	0	0	0	0000
JALR	I	-	000	1	1	1	1	0	0	0	0	0	1	0011
LB		-	000	1	1	0	0	0	1	0	1	0	1	0011
LH		-	001	1	1	0	0	0	1	0	1	0	1	0011
LW		-	010	1	1	0	0	0	1	0	1	0	1	0011
LBU		-	100	1	1	0	0	0	1	0	1	0	1	0011
LHU		-	101	1	1	0	0	0	1	0	1	0	1	0011
ADDI		-	000	1	1	0	0	0	0	0	0	0	1	0011
SLTI		-	010	1	1	0	0	0	0	0	0	0	1	0101
SLTIU		-	011	1	1	0	0	0	0	0	0	0	1	1100
XORI		-	100	1	1	0	0	0	0	0	0	0	1	0010
ORI		-	110	1	1	0	0	0	0	0	0	0	1	0001
ANDI		-	111	1	1	0	0	0	0	0	0	0	1	0000
LLI		-	001	1	1	0	0	0	0	0	0	0	1	0110
SRLI		-	101	1	1	0	0	0	0	0	0	0	1	0111
SRAI		-	101	1	1	0	0	0	0	0	0	0	1	1000
FENCE		-	000	1	1	0	0	0	0	0	0	0	1	1111
FENCE.TSO		-	000	1	1	0	0	0	0	0	0	0	1	1111
PAUSE		-	000	1	1	0	0	0	0	0	0	0	1	1111
ECALL		-	000	1	1	0	0	0	0	0	0	0	1	1111
EBREAK		-	000	1	1	0	0	0	0	0	0	0	1	1111
SB	S	-	000	1	0	0	0	0	0	1	0	0	1	0011
SH		-	001	1	0	0	0	0	0	1	0	0	1	0011
SW		-	010	1	0	0	0	0	0	1	0	0	1	0011
BEQ	B	-	000	1	0	0	0	1	0	0	0	0	0	0010
BNE		-	001	1	0	0	0	1	0	0	0	0	0	0010
BLT		-	100	1	0	0	0	1	0	0	0	0	0	0101
BGE		-	101	1	0	0	0	1	0	0	0	0	0	0101
BLTU		-	110	1	0	0	0	1	0	0	0	0	0	1100
BGEU		-	111	1	0	0	0	1	0	0	0	0	0	1100
LUI	U	-	-	0	1	0	0	0	0	0	0	0	1	1111
AUIPC		-	-	0	1	0	0	0	0	0	0	1	1	0011
JAL	J	-	-	0	1	0	1	0	0	0	0	1	1	0011
-	Illegal	-	-	0	0	0	0	0	0	0	0	0	0	0000

Appendix C – Zircon Assembler v1.0.0

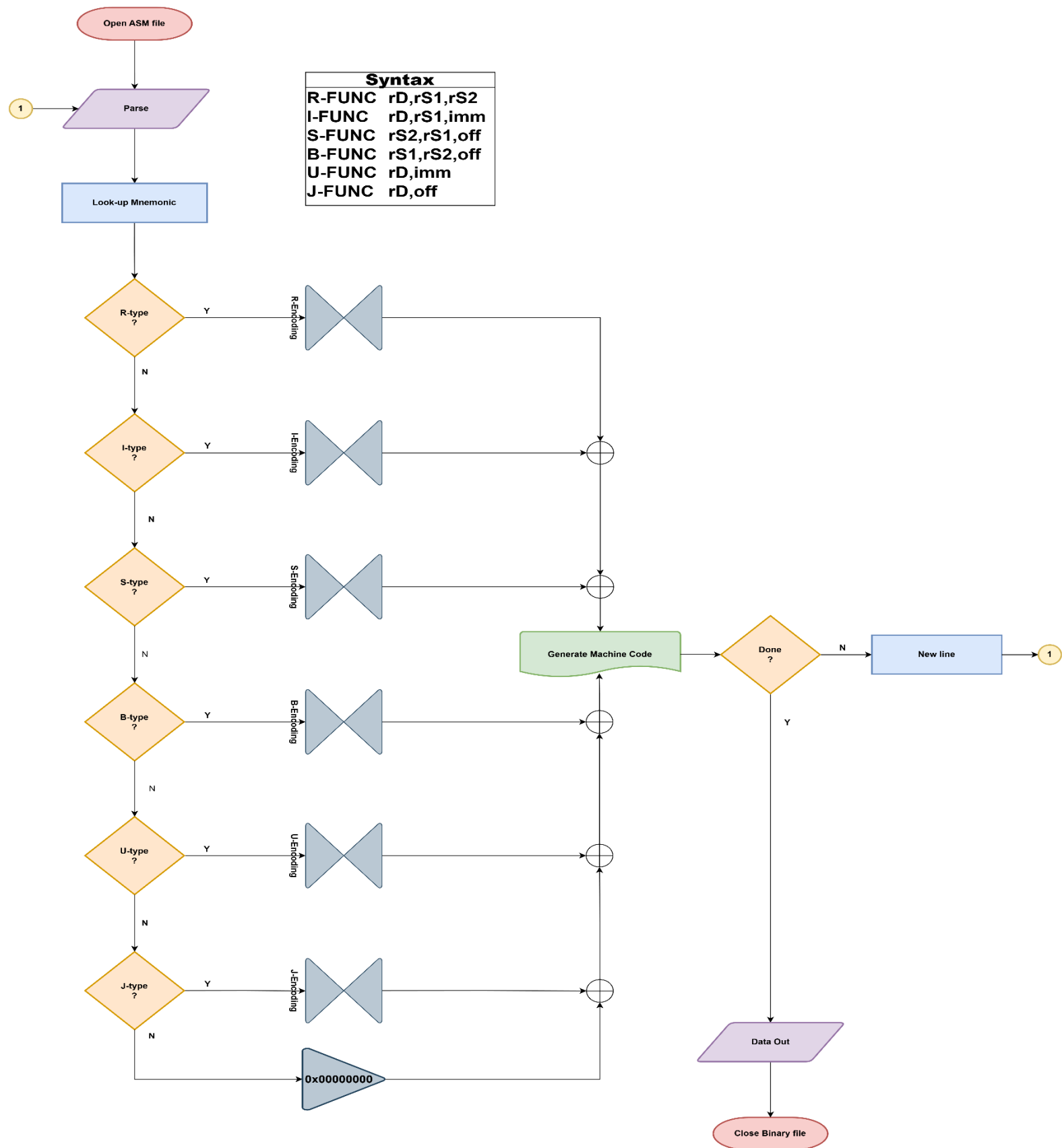


Figure C-1: ZASM v1.0.0 Algorithm